CSS 3

Cascading Style Sheet

- C’est un langage informatique qui permet de décrire et spécifier la présentation des documents HTML .

- Introduit au milieu des années 1990.
  Il a été bien pris en charge par les navigateurs à partir des années 2000.

- ❖ Styles définis à l’intérieur d’une balise html.
  ❖ Styles définis dans une balise <style> à l’intérieur du <head>.
  ❖ Styles définis dans un fichier css extérieur.

Cascading Style Sheet

- On insère du CSS au document HTML entre :

```
<style type=" text/css " >
   <!--  du css ici  -->
 <style>
```

- Ou On lie un fichier CSS avec:

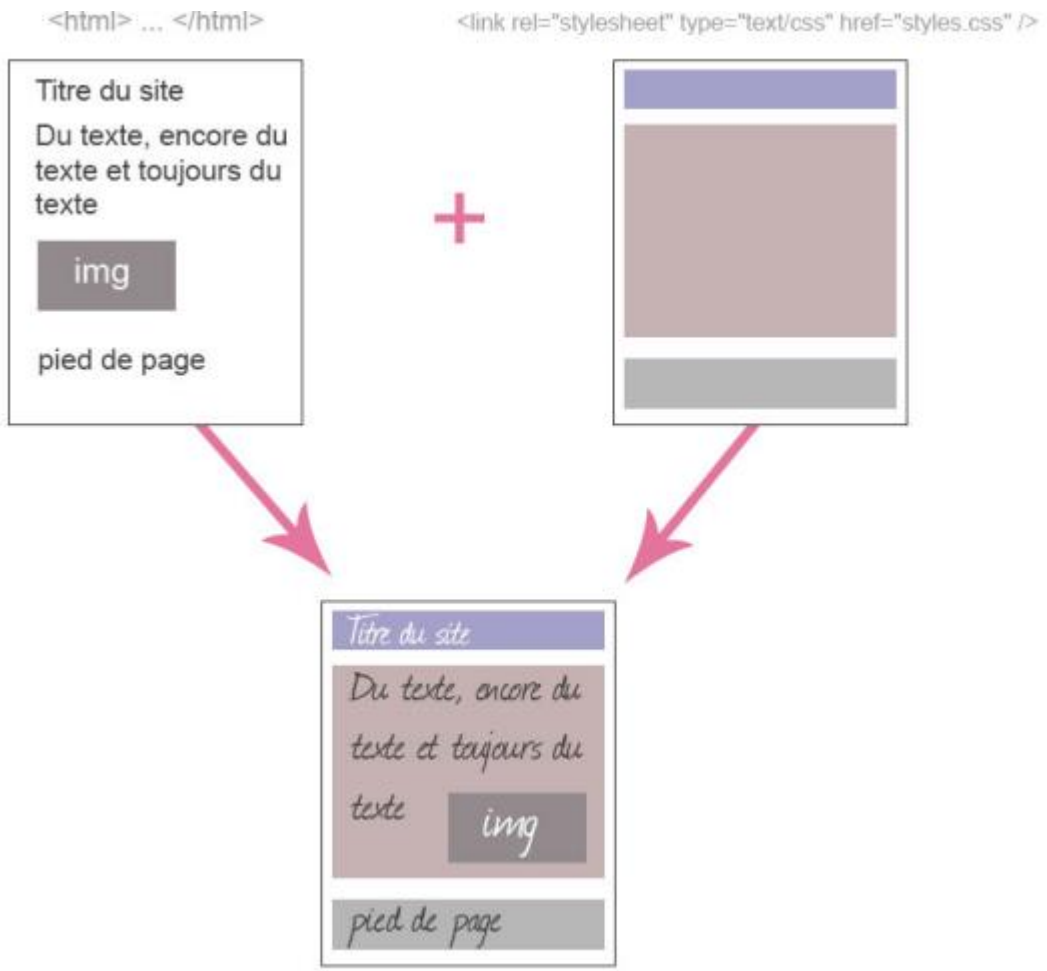
```
<link rel = "stylesheet "   type=  " text/css "   href= " nom_fichier.css " >
```

- Commentaires CSS: entre /* et */

Cascading Style Sheet

index.html : pour le contenu

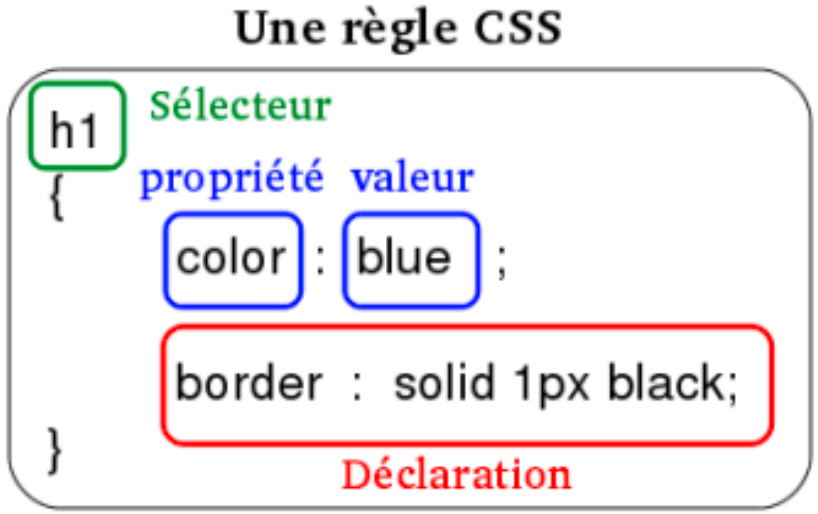
styles.css : pour la mise en page / présentation



Syntaxe CSS

Jeu de règles

- Un jeu de règles se compose d'un sélecteur suivi par un bloc de déclaration délimité par des accolades ({ }).
- Une déclaration est constituée d'une propriété, suivie du caractère deux-points (:) puis d'une valeur.



Syntaxe CSS

Les sélecteurs

Un sélecteur est un motif de reconnaissance permettant d'appliquer un style aux éléments de l'arbre du document HTML.

1. À tous les éléments de la page : le sélecteur universel ***** agit sur tous les éléments HTML.

```
* { color: #ff0000; }

  /* pour définir une couleur rouge par défaut */
```

Syntaxe CSS

Les sélecteurs

2. À toutes les instances d'un élément : les sélecteurs de type agissent sur un type d'élément HTML.

```
p { color: #0000ff; }

/* définir une couleur bleue pour tous les textes des paragraphes*/
```

Syntaxe CSS

Les sélecteurs

3. À différents éléments simultanément : le regroupement de sélecteurs

```
h3,h4,h5,h6 { color: brown; }

/* définir une couleur marron pour tous les titres de niveau 3, 4, 5 et 6*/
```

Syntaxe CSS

Les sélecteurs

4. À certaines instances d'un élément : les sélecteurs de classe

```
p.vert { color: #008000; }

/* définir une couleur verte pour les textes de certains paragraphes (ceux de la classe .vert) */
```

Le contenu de toutes les balises <p class=" vert "> sera vert. Et le contenu des balises <p> restera bleu.

Syntaxe CSS

Les sélecteurs

5. À une instance unique d'un élément : les sélecteurs d' **id**

```
p#gris { color: #7F7F7F; }

/* définir une couleur grise pour le texte d'un paragraphe précis */
```

Le contenu de la seule balise <p id="gris"> sera gris. Le contenu de toutes les balises <p class=" vert "> sera vert. Et le contenu des balises <p> restera bleu.

Syntaxe CSS

➡ Les sélecteurs

6. À un élément directement ou indirectement contenu dans un autre élément : les sélecteurs descendants

```
h2 strong { color: pink; }

/* définir une couleur rose d'un élément em contenu dans un élément h2*/
```

qui s'appliquera aussi bien à : <h2>en rose ...</h2>

qu'à : <h2><code>en rose</code>...</h2>

Syntaxe CSS

➡ Les sélecteurs

7. À un élément directement contenu dans un autre élément : les sélecteurs d'enfant

P > em { color: yellow; }

/* définir une couleur jaune d'un élément em descendant de l'élément p*/

qui s'appliquera seulement à : * en jaune ... *

mais pas à : * <code>en jaune</code>... *

Syntaxe CSS

Les sélecteurs

8. À un élément suivant un autre élément : les sélecteurs d'enfants adjacents

```
img + p { color: kakhi; }
```

```
/* définir une couleur kaki d'un élément p suivant directement un  img*/
```

qui s'appliquera seulement à : *<img>.....</img>* * en kaki *

mais pas à : *<h2>.....</h2>* * ..... *

Syntaxe CSS

Exemple

Un lien en rouge (sélecteur universel *)

Un paragraphe en bleu (sélecteur de type p)

Un paragraphe en vert (sélecteur de classe)

Un paragraphe en gris (sélecteur d'id)

Un titre de niveau 2 en rouge (sélecteur universel *) *et en rose (sélecteur descendant)*

Ceci sera en bleu (sélecteur de type p) ... *et ceci en jaune (sélecteur d'enfant)*

Un paragraphe en kaki (sélecteur d'enfant adjacent)

Un titre de niveau 3 en marron (regroupement)

Un titre de niveau 4 en marron (regroupement)

Un titre de niveau 5 en marron (regroupement)

Un titre de niveau 6 en marron (regroupement)

Unités de mesures

Les CSS proposent 8 unités pour exprimer tailles et longueurs.

❖Les unités absolues sont : le point (pt), le pica (pc), le centimètre (cm), le millimètre (mm) et le pouce (in). (1 in = 2,54cm = 25,4 mm = 72 pt = 6 pc).

❖ Les unités relatives sont : le « em » , le « ex » , le pourcentage (%) et le pixel (px). Les deux premières unités sont relatives à la police de référence : 1 em est égal à la taille de cette police, tandis que 1 ex est la hauteur dans cette police.

Les couleurs

- ✓**mot-clé :** black, blue, brown, cyan, gray, green, pink, purple, red, ...

- ✓**code hexadécimal** : #999999, composantes RGB

- ✓**fonction rgb() :** rgb(r,v,b) avec $0 \leq r, v, b \leq 255$

LES PROPRIETES CSS DE TYPE "FONT-"

➡ Font-size:

- Utilisé pour modifier la taille d'un texte.
- Accepte les valeurs : px, em, ex, %, pt



LES PROPRIETES CSS DE TYPE "FONT-"

➡ Font-style:

- Permet de fixer l'inclinaison d'un texte.
- Accepte les valeurs : Normal , Italic, Oblique, Inherit

HTML

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width">
  <title>JS Bin</title>
</head>
<body>
   <p>Bonjour</p>
   <p>Je suis en Italique</p>
   <div>
     <p class="normal">Et bien pas moi</p>
     <p class="normal">Moi non plus</p>
  </div>
</body>
</html>
```

CSS

```
p{
  font-style:italic;
}
div{
  font-style:normal;
}
.normal{
  font-style:inherit;
}
```

Output

Bonjour

Je suis en Italique

Et bien pas moi

Moi non plus

LES PROPRIETES CSS DE TYPE "FONT-"

➡ Font-weight:

- Permet de fixer l'inclinaison d'un texte.
- Accepte les valeurs : Normal, Lighter, Bolder, Bold,100-900, Inherit

HTML

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width">
  <title>JS Bin</title>
</head>
<body>
     <p class="p1">Je suis normal</p>
     <p class="p2">Je suis en gras</p>
</body>
</html>
```

CSS

```
.p1{
  font-weight:400;
}
.p2{
  font-weight:bold;
}
```

Output 456px

Je suis normal

Je suis en gras

LES PROPRIETES CSS DE TYPE "FONT-"

➡ line-height:

- sert à fixer l'écartement, c'est-à-dire la distance entre deux lignes de texte,

HTML

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width">
  <title>JS Bin</title>
</head>
<body>
     <p class="p1">Je suis normal</p>
     <p class="p2">Je suis en gras</p>
</body>
</html>
```

CSS

```
.p1{
  font-size:18px;
  font-weight:400;
  line-height:30px;
}
.p2{
  font-weight:bold;
}
```

Output

Je suis normal

Je suis en gras

LES PROPRIETES CSS DE TYPE "FONT-"

font-family:

- Permet de choisir la police de notre texte.
- La première police déclarée sera le choix par défaut.

HTML

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width">
  <title>JS Bin</title>
</head>
<body>
     <p class="p1">Je suis normal</p>
     <p class="p2">Je suis en gras</p>
</body>
</html>
```

CSS

```
.p1{
  font-size:18px;
  font-weight:400;
  line-height:30px;
  font-family:Tahoma, Calibri, Arial, sans-serif;
}
.p2{
  font-weight:bold;
}
```

Output

Je suis normal

Je suis en gras

LES PROPRIETES CSS DE TYPE "TEXT-"

➡ text-align:

- Permet de modifier l'alignement d'un texte.
- Accepte les valeurs : Left, right, center, Justify, inhérit

HTML

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width">
  <title>JS Bin</title>
</head>
<body>
     <p class="p1">Je suis normal</p>
     <p class="p2">Je suis en gras</p>
</body>
</html>
```

CSS

```
.p1{
   text-align:center;
}
.p2{
   text-align:right;
   font-weight:bold;
}
```



LES PROPRIETES CSS DE TYPE "TEXT-"

➡ text-decoration:

- Permet de modifier la décoration d'un texte.
- Accepte les valeurs : Underline, Overline, Line-through, Inherit, None

HTML

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width">
  <title>JS Bin</title>
</head>
<body>
  <div>
     <p class="pdiv">Un paragraphe dans un div</p>
  </div>
     <p >Un autre paragraphe</p>
</body>
</html>
```

CSS

```
p{
  text-align:center;
  text-decoration:underline;
}
.pdiv{
   text-decoration:line-through;
}
```

~~Un paragraphe dans un div~~

Un autre paragraphe

LES PROPRIETES CSS DE TYPE "TEXT-"

➡ text-transform:

- Permet de modifier l'aspect des caractères d'un texte,
- Accepte les valeurs : Uppercase, Lowercase, Capitalize, None, Inherit

HTML

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width">
  <title>JS Bin</title>
</head>
<body>
     <p >Paragraphe étalon</p>
     <p class="p1">Un paragraphe dans un div</p>
     <p class="p2">Un autre paragraphe</p>
</body>
</html>
```

CSS

```
.p1{
  text-transform:uppercase;
}
.p2{
  text-transform:capitalize;
}
```

Paragraphe étalon

UN PARAGRAPHE DANS UN DIV

Un Autre Paragraphe

LES PROPRIETES CSS DE TYPE "TEXT-"

➡ text-shadow:

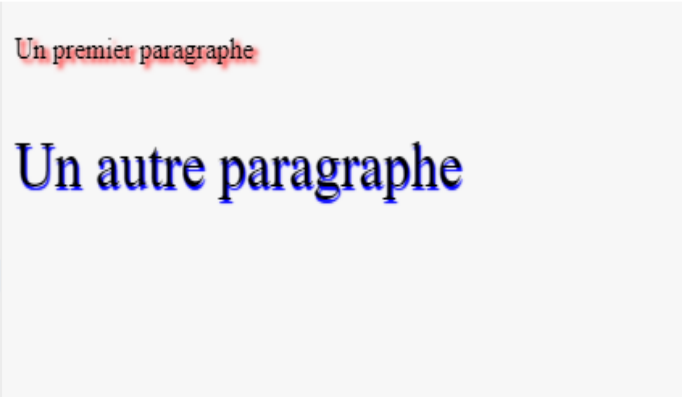
- Permet d'ajouter des effets d'ombre à un texte,
- Les 3 premières valeurs correspondent à des longueurs et la dernière est la couleur de l'ombre

HTML

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width">
  <title>JS Bin</title>
</head>
<body>
     <p class="p1">Un premier paragraphe </p>
     <p class="p2">Un autre paragraphe</p>
</body>
</html>
```

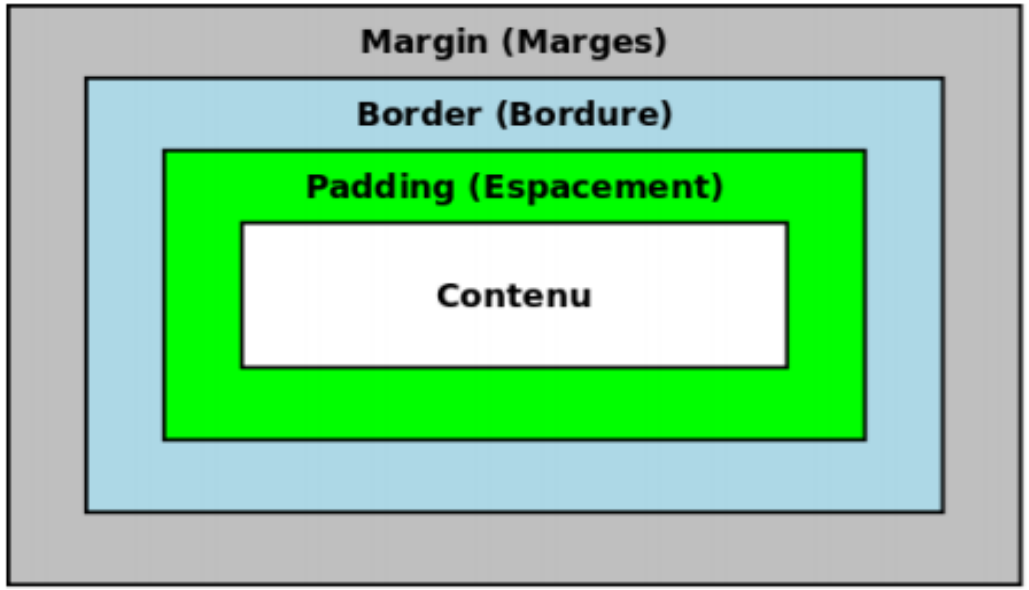
CSS

```
.p1{
  text-shadow:3px 2px 3px red;
}
.p2{
  font-size:34px;
  text-shadow:1px 2px 1px blue;
}
```



LE MODELE DES BOITES

- Tout élément d'une page est une boîte rectangulaire et peut avoir un padding, une marge et des bordures



LE MODELE DES BOITES

Hauteur et largeur:

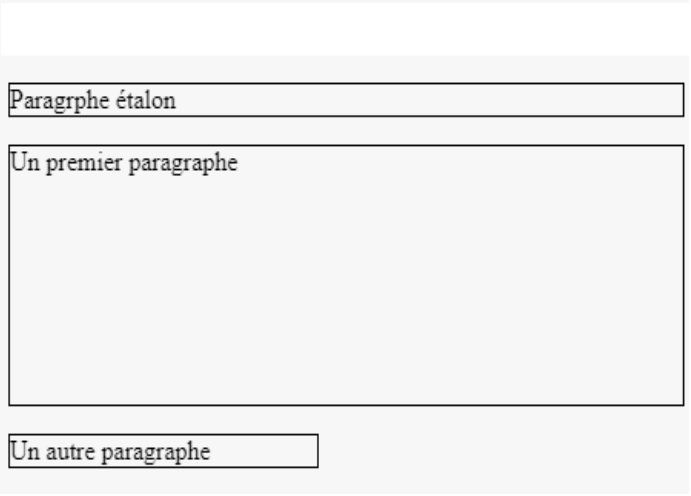
- Pour modifier la hauteur, on utilise la propriété height (px, %, auto),
- La propriété width permet de modifier la largeur,

HTML

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width">
  <title>JS Bin</title>
</head>
<body>
     <p> Paragrphe étalon </p>
     <p class="p1">Un premier paragraphe </p>
     <p class="p2">Un autre paragraphe</p>
</body>
</html>
```

CSS

```
p{
  border: 1px solid black;
}
.p1{
  height:150px;
}
.p2{
  width:200px;
}
```



LE MODELE DES BOITES

Les bordures:

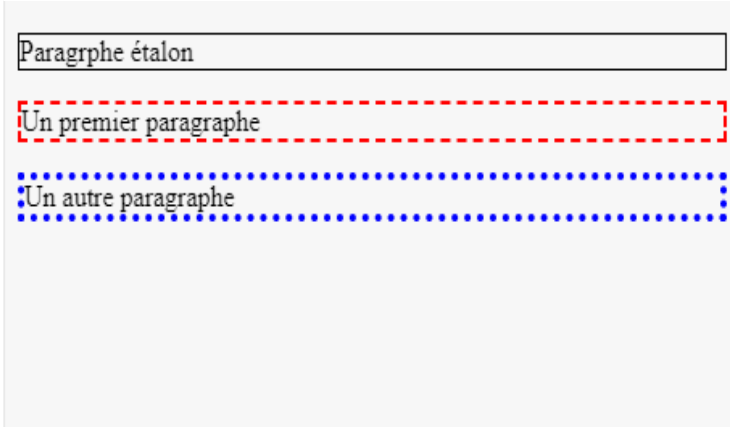
- Pour créer des bordures et les personnaliser, on a les 3 propriétés: border-width , border-style (solid, double, dashed, dotted), border-color,

HTML

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width">
  <title>JS Bin</title>
</head>
<body>
     <p> Paragrphe étalon </p>
     <p class="p1">Un premier paragraphe </p>
     <p class="p2">Un autre paragraphe</p>
</body>
</html>
```

CSS

```
p{
  border: 1px solid black;
}
.p1{
  border-width:2px;
  border-style:dashed;
  border-color:red;
}
.p2{
  border-width:4px;
  border-style:dotted;
  border-color:blue;
}
```



LE MODELE DES BOITES

➡ Les bordures:

- Pour créer des bordures arrondies, on utilise la propriété border-radius,
- Pour chaque côté, on peut définir les angles en utilisant les mots clefs:

  top-left, top-right, bottom-left et bottom-right.

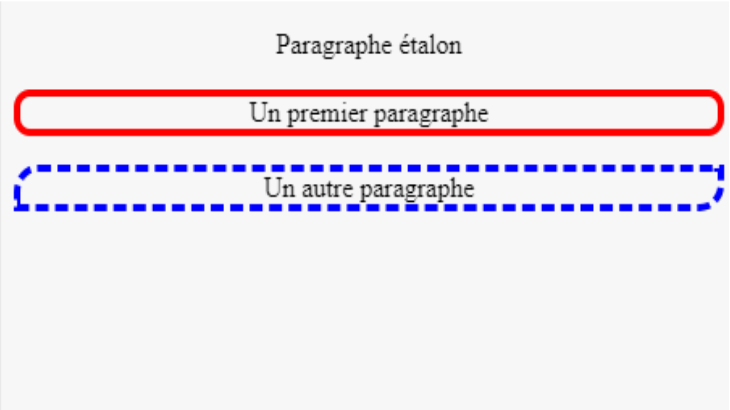
HTML

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width">
  <title>JS Bin</title>
</head>
<body>
     <p> Paragraphe étalon </p>
     <p class="p1">Un premier paragraphe </p>
     <p class="p2">Un autre paragraphe</p>
</body>
</html>
```

CSS

```
p{
  text-align:center;
  }
.p1{
  border: 4px solid red;
  border-radius:10px;
}
.p2{
  border: 4px dashed blue;
  border-top-left-radius:15px;
  border-bottom-right-radius:15px;

}
```



LE MODELE DES BOITES

➡ Les marges intérieures:

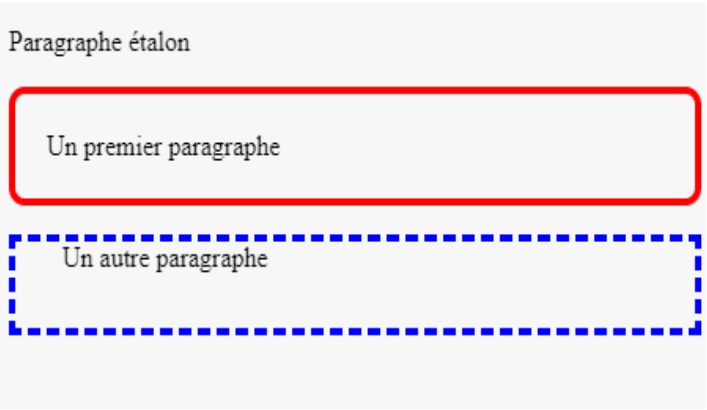
- La propriété padding va définir l'espace entre l'élément et sa bordure.
- On peut définir des espacements différents pour chaque marge intérieure en utilisant: padding-right, padding-bottom, padding-left et padding-top

HTML

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width">
  <title>JS Bin</title>
</head>
<body>
     <p> Paragraphe étalon </p>
     <p class="p1">Un premier paragraphe </p>
     <p class="p2">Un autre paragraphe</p>
</body>
</html>
```

CSS

```
.p1{
  border: 4px solid red;
  border-radius:10px;
  padding:20px;
}
.p2{
  border: 4px dashed blue;
  padding-left:30px;
  padding-bottom:30px;
}
```



LE MODELE DES BOITES

➡ Les marges extérieures:

- La propriété margin permet de définir la taille des marges extérieures,
- Tout comme la propriété padding, on peut définir des espacements différents pour chaque marge extérieure (right, bottom, left et top)

HTML

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width">
  <title>JS Bin</title>
</head>
<body>
     <p> Paragraphe étalon </p>
     <p class="p1">Un premier paragraphe </p>
     <p class="p2">Un autre paragraphe</p>
</body>
</html>
```

CSS

```
body{
  padding:0px;
  margin:0px;
}
.p1{
  border: 4px solid red;
  border-radius:10px;
  padding:20px;
  margin-bottom:70px;
}
.p2{
  border: 4px dashed blue;
  padding:30px;
  margin:20px;
}
```



LE MODELE DES BOITES

Les ombres des boîtes:

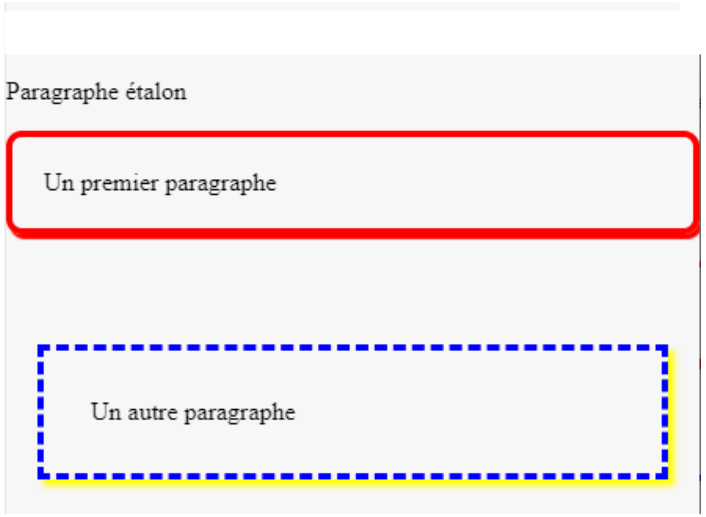
- On peut créer des ombres autour des boîtes avec la propriété box-shadow,
- Cette propriété fonctionne comme text-shadow (accepte 4 valeurs),

HTML

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width"
  <title>JS Bin</title>
</head>
<body>
     <p> Paragraphe étalon </p>
     <p class="p1">Un premier paragraphe </p>
     <p class="p2">Un autre paragraphe</p>
</body>
</html>
```

CSS

```
body{
  padding:0px;
  margin:0px;
}
.p1{
  border: 4px solid red;
  border-radius:10px;
  padding:20px;
  margin-bottom:70px;
  box-shadow:2px 3px 1px red;
}
.p2{
  border: 4px dashed blue;
  padding:30px;
  margin:20px;
  box-shadow:4px 4px 4px yellow;
}
```



LE MODELE DES BOITES

➡ Faire flotter un élément:

- Pour aligner des éléments les uns par rapport aux autres, on peut les faire « flotter » en utilisant la propriété float avec les valeurs: left, right, none ou inherit,

HTML

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width">
  <title>JS Bin</title>
</head>
<body>
     <p> Paragraphe étalon </p>
     <p class="p1">Un premier paragraphe </p>
  <p class="p2">Un <strong>autre</strong> paragraphe</
</body>
</html>
```

CSS

```
body{
  padding:0px;
  margin:0px;
}
.p1{
  border: 4px solid red;
  border-radius:10px;
  padding:20px;
  float:left;
  box-shadow:2px 3px 1px red;
}
.p2{
  border: 4px dashed blue;
  padding:30px;
  margin:20px;
  box-shadow:4px 4px 4px yellow;
  float:right;
}
strong{
  float:right;
}
```



Exercice 1:

Écrire le code HTML et la feuille de style CSS pour obtenir le rendu suivant, en utilisant les codes couleurs ci-après.

- Jaune : #FCD116
- Bleu : #003893
- Rouge : #CE1126

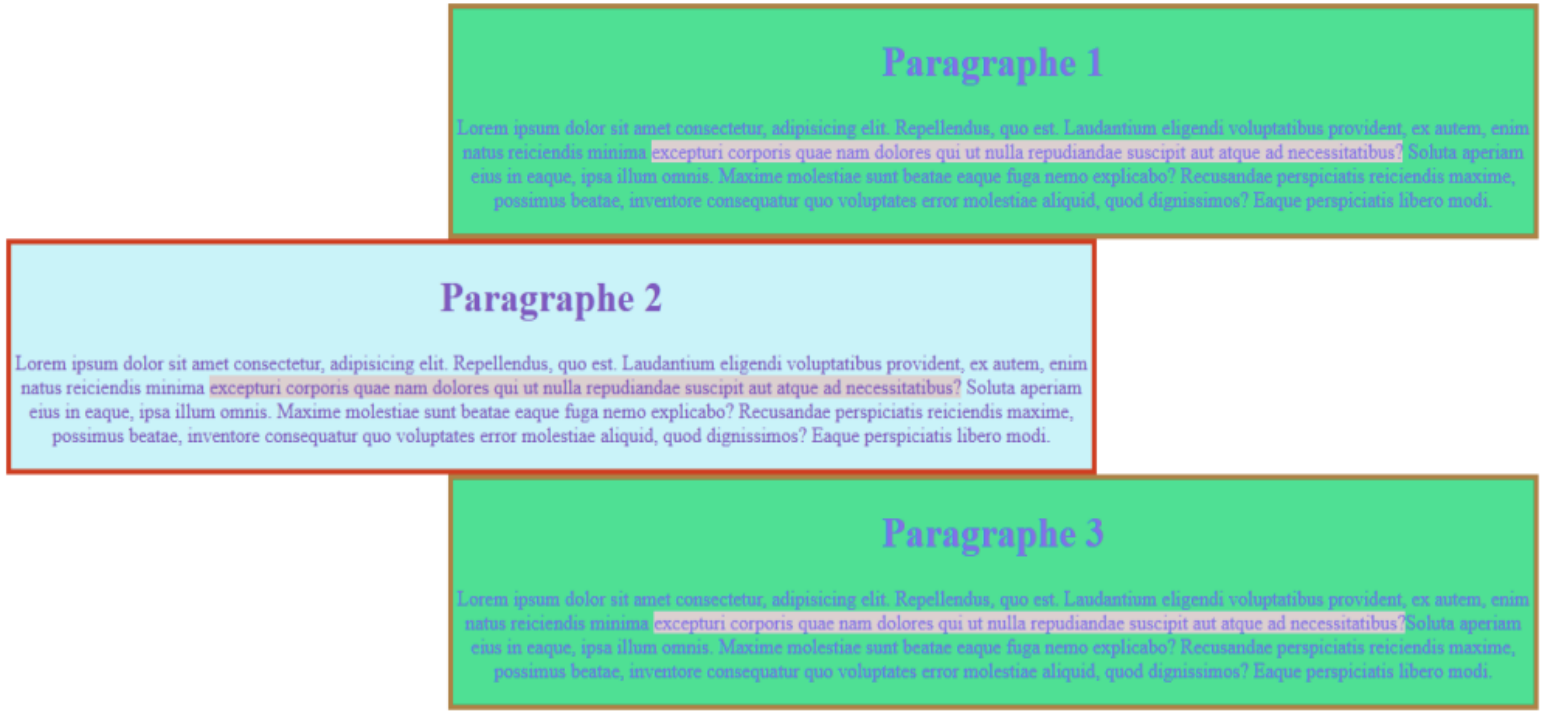


Colombie

Capitale : Bogota - **Indépendance** : 7 août 1819 - **Population** : 49 084 841 habitants

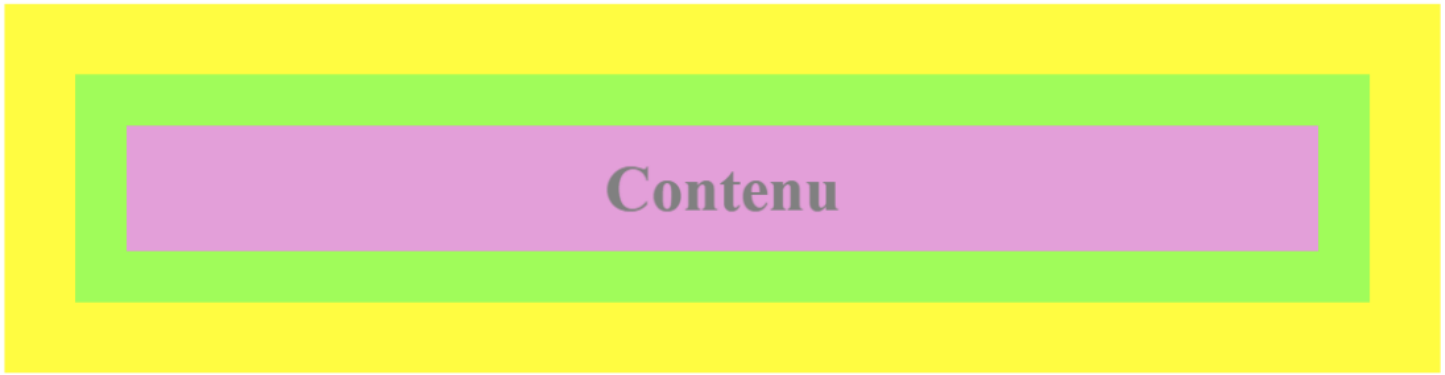
Exercice 2:

Réaliser un document HTML et CSS permettant d'afficher:



Exercice 3:

Utiliser les propriétés padding, margin et border pour afficher cette apparence :



Position

Positionnement relatif :

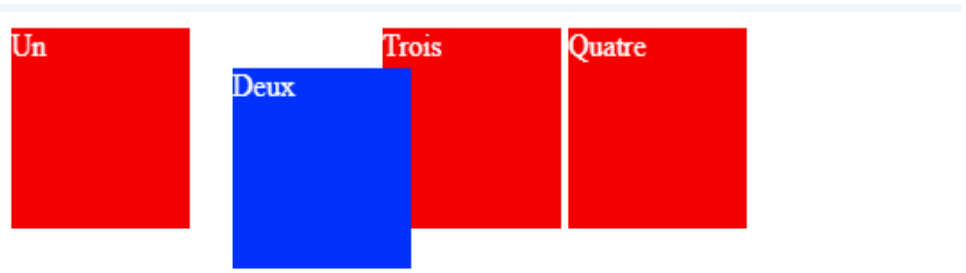
- Un élément positionné de façon relative est un élément dont la propriété de position calculée est relative. Dans ce cas, les propriétés **top** ou **bottom** indiquent le décalage vertical à appliquer et **left** ou **right** indiquent le décalage horizontal.

HTML

```html
<div class="box" id="un">Un</div>
<div class="box" id="deux">Deux</div>
<div class="box" id="trois">Trois</div>
<div class="box" id="quatre">Quatre</div>
```

CSS

```css
.box {
 display:inline-block;
 width: 100px;
 height: 100px;
 background: red;
 color: white;
}
#deux {
 position: relative;
 top: 20px;
 left: 20px;
 background: blue;
}
```



Position

Positionnement absolu :

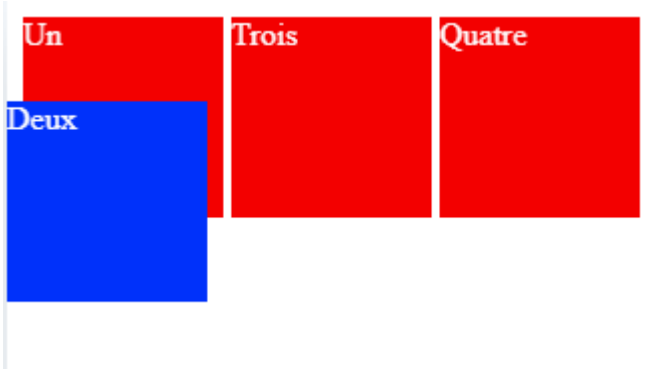
- Un élément positionné de façon absolue est un élément dont la propriété de position calculée est **absolute**. Dans ce cas, les propriétés **top**, **bottom**, **right** et **left** indiquent les distances entre les bords de l'élément et les bords du bloc englobant

HTML

```html
<div class="box" id="un">Un</div>
<div class="box" id="deux">Deux</div>
<div class="box" id="trois">Trois</div>
<div class="box" id="quatre">Quatre</div>
```

CSS

```css
.box {
  display:inline-block;
  width: 100px;
  height: 100px;
  background: red;
  color: white;
}
#deux {
  position: absolute;
  top: 50px;
  left: 0px;
  background: blue;
}
```



Position

Positionnement fixe :

- On utilise souvent ce positionnement pour créer un élément flottant qui reste à la même position, même lorsqu'on fait défiler la page.

HTML

```html
<div class="outer">
 <p> Lorem ipsum dolor sit amet, consectetur adipiscing elit.
Nam congue tortor eget pulvinar lobortis. Vestibulum ante
ipsum primis in faucibus orci luctus et ultrices posuere
cubilia Curae; Nam ac dolor augue. Pellentesque mi mi,
laoreet et dolor sit amet, ultrices varius risus. Nam vitae iaculis
elit. Aliquam mollis interdum libero. Sed sodales placerat
egestas. Vestibulum ut arcu aliquam purus viverra dictum vel
sit amet mi. </p>
<p> Lorem ipsum dolor sit amet, consectetur adipiscing elit.
Nam congue tortor eget pulvinar lobortis. Vestibulum ante
ipsum primis in faucibus orci luctus et ultrices posuere
cubilia Curae; Nam ac dolor augue. Pellentesque mi mi,
laoreet et dolor sit amet, ultrices varius risus. Nam vitae
iaculis elit. Aliquam mollis interdum libero. Sed sodales
placerat egestas.  </p>
 <div class="box" id="un">Un</div>
</div>
```

CSS

```css
.box {
 width: 100px;
 height: 100px;
 background: red;
 color: white;
}
#un {
 position: fixed;
 top: 80px;
 left: 10px;
 background: blue;
}
.outer {
 width: 500px;
 height: 200px;
 overflow: scroll;
 padding-left: 150px;
}
```



Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nam congue tortor eget pulvinar lobortis. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Nam ac dolor augue. Pellentesque mi mi, laoreet et dolor sit amet, ultrices varius risus. Nam vitae iaculis elit. Aliquam mollis interdum libero. Sed sodales placerat egestas. Vestibulum ut arcu aliquam purus viverra dictum vel sit amet mi.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nam congue tortor eget pulvinar lobortis. Vestibulum ante ipsum primis in faucibus orci luctus

Position

Positionnement adhérent (Sticky) :

- Le positionnement adhérent est un mélange de positionnement relatif et de positionnement fixe. L'élément est considéré comme positionné de façon relative, jusqu'à ce qu'un seuil soit franchi. À partir de ce seuil, l'élément est positionné de façon fixe.

HTML

```html
<dl>
 <div>
 <dt>A</dt> <dd>Andrew W.K.</dd> <dd>Apparat</dd> <dd>Arcade Fire</dd>
 <dd>At The Drive-In</dd> <dd>Aziz Ansari</dd>
 </div>
 <div>
 <dt>C</dt> <dd>Chromeo</dd> <dd>Common</dd> <dd>Converge</dd>
 <dd>Crystal Castles</dd> <dd>Cursive</dd>
 </div>
 <div>
 <dt>E</dt> <dd>Explosions In The Sky</dd>
 </div>
 <div>
 <dt>T</dt> <dd>Ted Leo & The Pharmacists</dd> <dd>T-Pain</dd>
 <dd>Thrice</dd> <dd>TV On The Radio</dd> <dd>Two Gallants</dd>
 </div>
</dl>
```

CSS

```css
.dl > div { background: #fff; padding: 24px 0 0 0;}
dt { background: #b8c1c8; border-bottom: 1px solid #989ea4;
 border-top: 1px solid #717d85; color: #fff;
 font: bold 18px/21px Helvetica, Arial, sans-serif;
 margin: 0; padding: 2px 0 0 12px; position: sticky;
 top: -1px;
}
dd {
 font: bold 20px/45px Helvetica,
 Arial, sans-serif;
 margin: 0;
 padding: 0 0 0 12px;
 white-space: nowrap;
}
dd + dd {
 border-top: 1px solid #ccc;
}
```

A
Andrew W.K.
Apparat
Arcade Fire
At The Drive-In
Aziz Ansari
C
Chromeo
Common

Flexbox

flex-direction

Comme son nom l'indique, la valeur de cette propriété définit la direction qu'auront ses enfants.

Valeurs possibles:

- row → *(par défaut)*
- row-reverse ←
- column ↓
- column-reverse ↑

Exemple

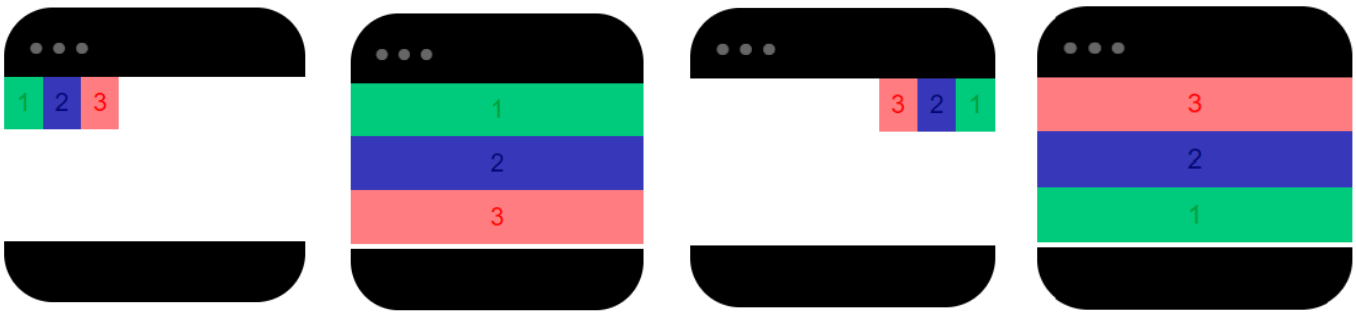
```
<div >

  <div class="frame">

    <div class="flex">
      <div class="box" style="background-color:green;">1</div>
      <div class="box" style="background-color:blue;">2</div>
      <div class="box" style="background-color:red;">3</div>
    </div>

  </div>
</div>
```

```
.flex {
  display: flex;
  flex-direction: row;
}
.box {
   Text-align :center ;
   height:20px;
  padding:30px;
}
```



justify-content

Un peu comme Word ou Google Doc, flexbox vous permet de justifier votre contenu horizontalement ↔ afin atteindre l'affichage désiré.

Possibilités:

- flex-start *(par défaut)*
- flex-end
- center
- space-between
- space-around
- space-evenly
- etc.

Exemple

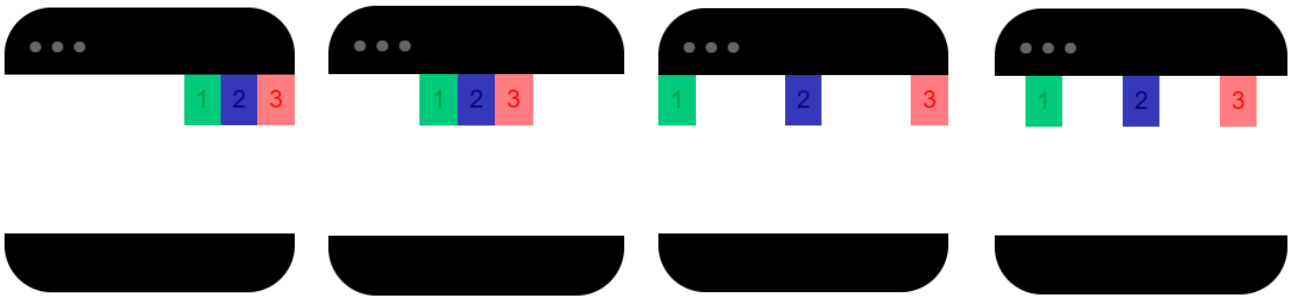
```
<div >

  <div class="frame">

    <div class="flex">
      <div class="box" style="background-color:green;">1</div>
      <div class="box" style="background-color:blue;">2</div>
      <div class="box" style="background-color:red;">3</div>
    </div>

  </div>
</div>
```

```
.flex {
  display: flex;
 justify-content : flex-end ;
}
.box {
   Text-align :center ;
   height:20px;
  padding:30px;
}
```



align-items

Dans la même lignée que justify-content, mais cette fois à la verticale [↕].

Possibilités:

- stretch *(par défaut)*
- flex-start
- flex-end
- center
- etc.

Exemple

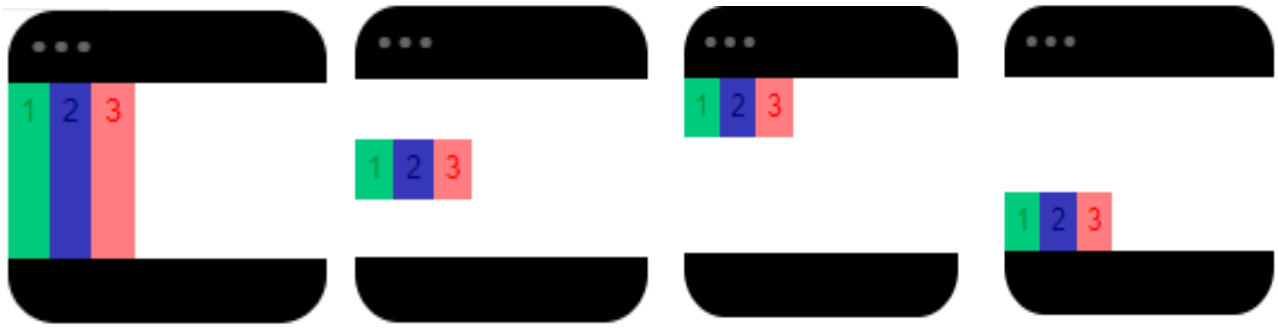
```
<div >

  <div class="frame">

    <div class="flex">
      <div class="box" style="background-color:green;">1</div>
      <div class="box" style="background-color:blue;">2</div>
      <div class="box" style="background-color:red;">3</div>
    </div>

  </div>
</div>
```

```
.flex {
  display: flex;
 align-items: stretch;
 height:100%;
}
.box {
   Text-align :center ;
  padding:30px;
}
```



align-self

La propriété align-self est pratiquement identique à la propriété align-items à la différence qu'elle s'applique sur un élément en particulier, **et non le parent**, afin de l'aligner de façon différente aux autres.

Elle accepte aussi les valeurs:

- stretch
- flex-start
- flex-end
- center
- etc.

Exemple

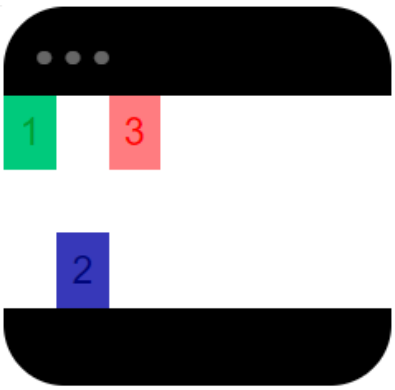
```
<div >

  <div class="frame">

    <div class="flex">
      <div class="box" style="background-color:green;">1</div>
      <div class="box" style="background-color:blue;
                              align-self :flex-end ;">2</div>
      <div class="box" style="background-color:red;">3</div>
    </div>

  </div>
</div>
```

```
.flex {
  display: flex;
  align-items: flex-start;
  height:100%;
}
.box {
  Text-align :center ;
  padding:30px;
}
```



flex-grow

Propriété pouvant être appliquée à un enfant dont le parent est en `display: flex;`.

Cette propriété accepte une valeur sans unité *(bref, un chiffre)* indiquant à l'élément qu'il peut grandir, quitte à dépasser son `width`, afin de remplir l'espace vacant dans son parent.

Par défaut, cette valeur est à `0`.

Exemple

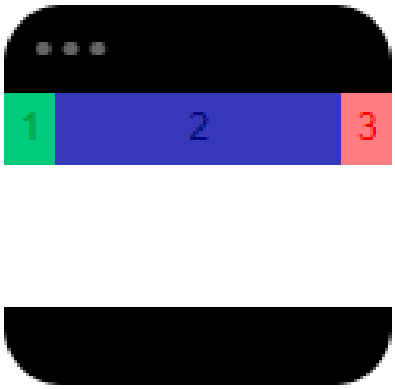
```
<div >

  <div class="frame">

    <div class="flex">
      <div class="box" style="background-color:green;">1</div>
      <div class="box" style="background-color:blue;
                                Flex-grow :2 ;">2</div>
      <div class="box" style="background-color:red;">3</div>
     </div>

  </div>
</div>
```

```
.flex {
  display: flex;
  }
.box {
  Text-align :center ;
  padding:30px;
}
```



Multiple flex-grow

Lorsque plusieurs éléments ont la propriété flex-grow, ils se partagent l'espace restant en fonction de la valeur qui leur est attribuée de façon proportionnelle.

Par exemple, s'ils ont tous la même valeur, ils grandiront tous de façon égale, tandis que si un élément à une valeur trois fois plus grande, il prendra trois fois plus d'espace que les autres.

```
<div >

  <div class="frame">

    <div class="flex">
      <div class="box" style="background-color:green; ">1</div>
      <div class="box" style="background-color:blue;
                                Flex-grow :3 ;">2</div>
      <div class="box" style="background-color:red;">3</div>
    </div>

  </div>
</div>
```

```
.flex {
  display: flex;
  }
.box {
  Flex-grow :1 ;
  Text-align :center ;
  padding:30px;
}
```



flex-shrink

Propriété pouvant être appliquée à un enfant dont le parent est en display: flex;.

Cette propriété accepte une valeur sans unité *(bref, un chiffre)* indiquant la proportion avec laquelle l'élément devrait rétrécir en cas de besoin *(si l'espace est insuffisant dans le parent)*. Bref, elle agit à l'inverse de flex-grow;

Par défaut, cette valeur est à 1.

```
<div >

  <div class="frame">


    <div class="flex">
      <div class="box" style="background-color:green; ">1</div>
      <div class="box" style="background-color:blue ;
                                  flex-shrink :0 ; ">2</div>
      <div class="box" style="background-color:red;">3</div>
    </div>

  </div>
</div>
```

```
.flex {
  display: flex;
  }
.box {
  width :50% ;
  Flex-shrink :1 ;
  Text-align :center ;
  padding:30px;
}
```





flex-basis

Propriété pouvant être appliquée à un enfant dont le parent est en display: flex;.

Cette propriété accepte n'importe quelle unité de mesure CSS afin d'indiquer la dimension idéale d'un élément.

Si la propriété flex-basis est spécifiée sur un élément, la propriété width est ignorée.

Par défaut, cette valeur est à auto.

```
<div >

  <div class="frame">

    <div class="flex">
      <div class="box" style="background-color:green;
                              Flex-basis :80% ;">1</div>
      <div class="box" style="background-color:blue ; ">2</div>
      <div class="box" style="background-color:red;">3</div>
    </div>

  </div>
</div>
```

```
.flex {
  display: flex;
  }
.box {
  Text-align :center ;
  padding:30px;
}
```



order

Définis la priorité d'affichage d'un enfant dans un parent ayant la propriété `display: flex;`.

Par défaut, cette valeur est à `0`.

Par défaut tous les éléments ont la même valeur d'ordre, ils s'affichent donc en fonction de leur ordre d'apparition dans le code, puisqu'aucun élément n'est jugé plus prioritaire qu'un autre. Cependant, si un élément à un ordre inférieur à 0 ▼, il se positionnera avant tous ceux ayant la valeur par défaut de 0. À l'inverse, si un élément à un ordre est supérieur à 0 ▲, il se positionnera après tous ceux ayant la valeur par défaut de 0.

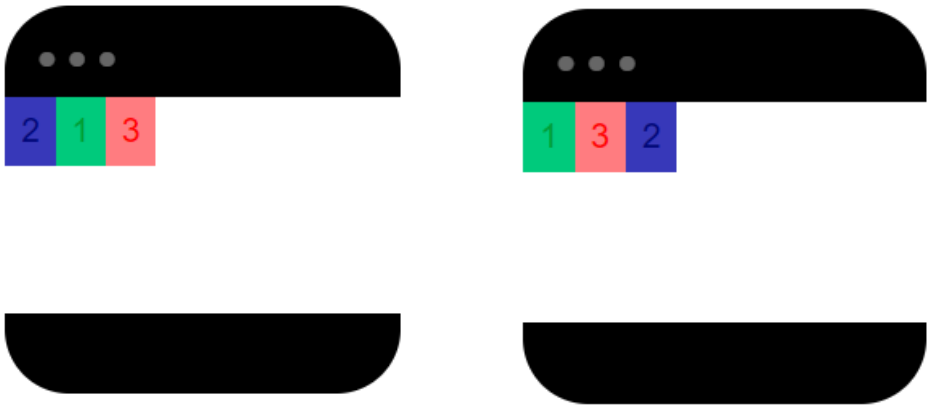
```
<div >

  <div class="frame">

    <div class="flex">
      <div class="box" style="background-color:green; ">1</div>
      <div class="box" style="background-color:blue ;
                                  order : -1 ; ">2</div>
      <div class="box" style="background-color:red;">3</div>
    </div>

  </div>
</div>
```

```
.flex {
  display: flex;
  }
.box {
  Text-align :center ;
  padding:30px;
}
```



flex

Cette propriété permet de définir le flex-grow, flex-shrink et flex-basis d'un élément en une seule propriété. Il s'agit en fait d'une propriété raccourcis.

Par exemple, si un élément à les propriétés:

flex-grow: 1;
flex-shrink: 0;
flex-basis: 25%;
Il est possible de faire l'équivalent simplement via:
flex: 1 0 25%;

```
<div >

  <div class="frame">

    <div class="flex">
      <div class="box" style="background-color:green; ">1</div>
      <div class="box" style="background-color:blue ; ">2</div>
      <div class="box" style="background-color:red;">3</div>
    </div>

  </div>
</div>
```

```
.flex {
  display: flex;
  }
.box {
   Flex : 1 0 25% ;
  Text-align :center ;
  padding:30px;
}
```



flex-wrap

Par défaut, flex essaie de tout afficher sur une même ligne.

Cependant, il est possible de lui spécifier d'afficher sur plus d'une ligne au besoin grâce à la propriété flex-wrap.

Valeurs possibles:

nowrap (Par défaut)



wrap



wrap-reverse



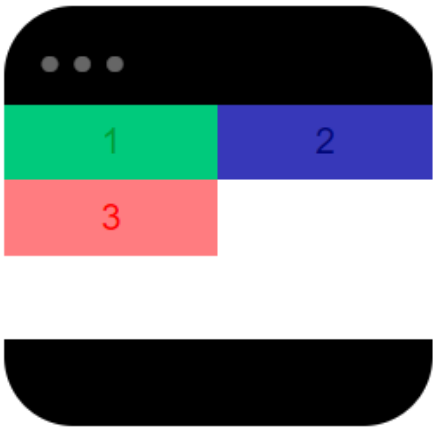
```
<div >

  <div class="frame">

    <div class="flex">
      <div class="box" style="background-color:green; ">1</div>
      <div class="box" style="background-color:blue ; ">2</div>
      <div class="box" style="background-color:red;">3</div>
    </div>

  </div>
</div>
```

```
.flex {
  display: flex;
  flex-wrap: wrap;
  }
.box {
  width :50% ;
  Text-align :center ;
  padding:30px;
}
```



flex-flow

Cette propriété permet de définir le flex-direction et flex-wrap en une seule propriété. Il s'agit d'une propriété raccourcis.

Par exemple:

flex-direction: row-reverse;
flex-wrap: wrap;

est équivalent à

flex-flow: row-reverse wrap;

```
<div >

  <div class="frame">

    <div class="flex">
      <div class="box" style="background-color:green; ">1</div>
      <div class="box" style="background-color:blue ; ">2</div>
      <div class="box" style="background-color:red;">3</div>
    </div>

  </div>
</div>
```

```
.flex {
  display: flex;
  flex-flow: row-reverse wrap;
  }
.box {
  width :50% ;
  Text-align :center ;
  padding:30px;
}
```



Exercice: https://flexboxfroggy.com/#fr

Spécificité

Chaque sélecteur en CSS se voit attribuer un pointage. Lorsque deux sélecteurs ciblent les mêmes propriétés d'un même élément, le sélecteur avec le plus haut pointage l'emporte. Le style de sa propriété est donc attribué à l'élément ciblé au détriment de celui du sélecteur plus faible.

Pointage par sélecteurs

1. **Universel ***

   Le sélecteur universel * est le plus faible des sélecteurs avec une valeur de **0 point**. Il peut donc être écrasé par tout autre sélecteur.

   ```
   * { ... }
   ```

2. **Balise**

   Un sélecteur avec une balise à une valeur de **1 point**.

   ```
   div { ... }
   ```

3. **Classe, pseudo-classe et attribut**

   Ont tous une valeur de **10 points** de spécificité.

- Classe

  ```
  .element { ... }
  ```

- Pseudo-classe

  ```
  :hover { ... }
  ```

  Normalement, une pseudo-classe est combinée à un autre sélecteur et n'est pas utilisé seul.

- Attribut

  ```
  [title="demo"] { ... }
  ```

- **ID**
  Un sélecteur avec un ID à une valeur de **100 points**.
  #no1 { ... }

- **Style en ligne**

  Les styles en ligne ont une valeur de **1 000 points**, ils sont donc extrêmement difficiles à écraser en cas de besoin, d'où pourquoi leur usage est à éviter autant que possible.
  `<div style="color: blue;">...</div>`

- **!important**

  En dernier recours, il est possible d'ajouter la mention `!important` à la fin d'une déclaration CSS afin de lui attribuer **10 000 points** de spécificité et ainsi s'assurer d'écraser tout autre style.

```
.element {
  color: blue !important; /* 10 000 points */
  background: red; /* 10 points */
}
```

  Puisque `!important` est ajouté à une propriété directement, sa présence n'impacte que la valeur de cette propriété et non celle de l'ensemble de la règle CSS.

  La présence d'un `!important` dans un code est généralement mal perçu et synonyme de problème de structure de code.

Combinaison de sélecteurs

La valeur de chaque sélecteur s'additionne dans un groupe de sélecteurs.

Par exemple l'élément HTML suivant:

- **Balise seule**

  Ciblé avec uniquement sa balise
  div { ... }
  La balise ne vaut que **1 point**.

- **Classe seule**

  .item { ... }
  La classe seule ne vaut que **10 points**.

- **Balise & Classe**

  mais combiné la balise et une classe
  div.item { ... }
  Donne au groupe de sélecteur une valeur de **11 points**.

- **Classe & Classe**

  Combiner les 2 classes
  .item.no1 { ... }
  Donne une valeur de **20 points**.

- **Balise & Classe & Classe**

  Et finalement combiner la balise aux deux classes
  div.item.no1 { ... }
  Donne **21 points**.

Égalité de pointage

Lorsque deux sélecteurs ont un pointage identique.
Par exemple pour l'élément HTML suivant:

```
<div class="element no1">
```

Les sélecteurs ci-dessous auront tous deux une valeur de spécificité équivalente à **10 points**.

```
.element { ... }
.no1 { ... }
```

Cependant, les styles du sélecteur `.no1` auront priorité sur ceux de `.element` puisque `.no1` est écrit en dernier et est donc considéré comme étant la définition la plus récente, donc celle prioritaire en cas d'égalité.

Exemples :

div ul ul li : Quatre descripteurs d'éléments = **4 points**
div.aside ul li : Un descripteur de classe, trois descripteurs d'éléments = **... points**
a:hover : Un descripteur de pseudo-classe, un descripteur d'élément = **... points**
div.navlinks a:hover :
#title em :

Site Calculateur Spécificité: https://specificity.keegan.st/

Exercice: https://fr.khanacademy.org/computing/computer-programming/html-css/more-css-selectors/e/quiz--css-specificity-rules